Django Permission System + Race Condition

Part 1: Django Permission System

1. What is Permission?

A Permission in Django determines whether a user is allowed to perform a specific action.

- **Authentication:** Who are you?
- **Authorization:** What are you allowed to do?
- **Permission:** What specific action can you perform?

Example: A user can view, create, and edit articles but cannot delete them.

2. Django's Built-in Model Permissions

When you create a Django model, Django normally creates four default permissions:

- `add`
- `change`
- `delete`
- `view`

Example:

```
# models.py
from django.db import models

class Article(models.Model):
    title = models.CharField(max_length=200)
```

Django creates permissions similar to:

```
blog.add_article
blog.change_article
blog.delete_article
blog.view_article
```

The full permission name is generally:
`app_label.codename`

3. Creating Custom Permissions

You can create custom permissions using the model's `Meta` class.

```
from django.db import models

class Article(models.Model):
```

```
    title = models.CharField(max_length=200)
    content = models.TextField()
    published = models.BooleanField(default=False)

    class Meta:
        permissions = [
            ("can_publish", "Can publish articles"),
        ]
```

Run:

```
python manage.py makemigrations
python manage.py migrate
```

Django creates the custom permission:

```
blog.can_publish
```

4. Manually Creating a Permission

You can also create permissions using Django's `Permission` model.

```
from django.contrib.auth.models import Permission
from django.contrib.contenttypes.models import ContentType
from django.contrib.auth.models import User

content_type = ContentType.objects.get_for_model(User)

permission = Permission.objects.create(
    codename="can_publish",
    name="Can publish articles",
    content_type=content_type,
)
```

For normal application-level custom permissions, defining them through `Meta.permissions` is usually cleaner and more maintainable.

5. Checking a User's Permission

The most important method is `user.has_perm()`.

```
if request.user.has_perm("blog.can_publish"):
    print("User can publish articles")
else:
    print("Permission denied")
```

6. Permission in Function-Based Views (FBV)

Django provides the `permission_required` decorator.

```
from django.contrib.auth.decorators import permission_required
from django.http import HttpResponse

@permission_required(
    "blog.can_publish",
    raise_exception=True,
)
def publish_article(request):
    return HttpResponse("Publish page")
```

`raise_exception=True` causes an authenticated user without the required permission to receive a permission-denied response.

7. Permission in Class-Based Views (CBV)

Use `PermissionRequiredMixin`.

```
from django.contrib.auth.mixins import PermissionRequiredMixin
from django.views.generic import TemplateView

class PublishView(PermissionRequiredMixin, TemplateView):
    template_name = "publish.html"
    permission_required = "blog.can_publish"
    raise_exception = True
```

URL configuration:

```
from django.urls import path
from .views import PublishView

urlpatterns = [
    path("publish/", PublishView.as_view(), name="publish"),
]
```

8. Permission Check in Templates

You can check permissions inside Django templates.

```
{% if perms.blog.can_publish %}
    <a href="{% url 'publish' %}">Publish Article</a>
{% endif %}
```

Important: Hiding a button or link is not a security mechanism. Users can still access the URL directly. Permissions must be enforced on the backend.

9. Admin Panel and Permissions

Django Admin allows you to manage:

- Users
- Groups
- Permissions

You can assign individual permissions to users through the admin panel.

10. Groups

Groups allow you to assign the same permissions to multiple users.

Example:

Editor Group

- `view_article`
- `add_article`
- `change_article`

When Faruk belongs to the Editor group, he receives the permissions assigned to that group.

11. User Permissions + Group Permissions

A user's effective permissions can come from both direct user permissions and group permissions.

Example:

User permissions:

- `delete_article`

Group permissions:

- `view_article`
- `change_article`

Effective permissions:

- `view_article`
- `change_article`
- `delete_article`

12. is_staff, is_superuser, and has_perm()

These are not the same.

is_staff

```
user.is_staff
```

Primarily determines whether a user can access Django Admin, subject to the applicable admin permissions.

is_superuser

```
user.is_superuser
```

A superuser has all permissions by default in Django's standard permission system.

has_perm()

```
user.has_perm("blog.can_publish")
```

Checks whether the user has a specific permission.

13. Permission Logic in utils.py

For complex business rules, you can create a helper function.

```
def can_access_publish(user):
    return (
        user.is_authenticated
        and user.is_verified
        and user.has_perm("blog.can_publish")
    )
```

Here, `is_verified` is an example of a custom user field. It must exist on your user model.

You can then use:

```
if can_access_publish(request.user):
    # Continue with the operation
    pass
```

The authorization check must still be enforced at the appropriate backend, view, or API layer.

14. Django Permission Flow

```
User Login
    |
Authentication
    |
User Identity
    |
Permission / Group Check
    |
Authorization
```

```
    |
Allow or Deny
```

Part 2: Race Condition

15. What is a Race Condition?

A Race Condition occurs when multiple concurrent operations access or modify the same shared data, and the result depends on the timing or order of those operations.

Example:

```
Multiple Requests
       |
Same Shared Data
       |
Concurrent Read / Write
       |
Race Condition
```

16. Simple Example

Suppose:

```
balance = 100
```

Two operations execute concurrently:

- Thread A subtracts 50.
- Thread B subtracts 30.

Expected result:

```
100 - 50 - 30 = 20
```

But both threads may read the original balance of 100.

```
Thread A reads 100
Thread B reads 100

Thread A calculates 100 - 50 = 50
Thread B calculates 100 - 30 = 70
```

If Thread B writes last, the final balance becomes 70 instead of 20.

This is a classic race condition.

17. Causes of Race Conditions

Common causes include:

1. **Concurrent execution:** Multiple threads, processes, or requests execute at the same time.
2. **Shared resources:** Database rows, files, memory variables, inventory, wallet balances, and counters.
3. **Non-atomic operations:** Reading a value, modifying it, and writing it back as separate operations.

Example:

```
value = obj.value
value -= 1
obj.value = value
```

This is a read-modify-write sequence rather than a single atomic database operation.

18. Race Condition in Django

Django applications can receive multiple requests at nearly the same time.

Example:

```
Product Stock = 10

Request A wants 7 items.
Request B wants 5 items.

Total requested = 12
Available stock = 10
```

Without concurrency protection, both requests may succeed based on stale stock values, causing overselling.

19. Unsafe Django Code

```
from django.http import JsonResponse
from .models import Product

def buy_product(request, product_id):
    product = Product.objects.get(id=product_id)
    qty = int(request.POST.get("qty", "0"))

    if product.stock >= qty:
        product.stock -= qty
        product.save()

        return JsonResponse({"status": "success"})
```

```
    return JsonResponse({"status": "out_of_stock"})
```

Problem: Two requests can read the same stock value before either saves its update. One update may overwrite the other, and both requests may incorrectly pass the stock check.

---

20. Solution 1 — select_for_update()

Django provides `select_for_update()` to request row-level locks within a transaction on supported database backends.

```
from django.db import transaction
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from .models import Product


@require_POST
def buy_product(request, product_id):
    try:
        qty = int(request.POST.get("qty", "0"))
    except (TypeError, ValueError):
        return JsonResponse(
            {"status": "invalid_quantity"},
            status=400,
        )

    if qty <= 0:
        return JsonResponse(
            {"status": "invalid_quantity"},
            status=400,
        )

    with transaction.atomic():
        product = (
            Product.objects
            .select_for_update()
            .filter(id=product_id)
            .first()
        )

        if product is None:
```

```
            return JsonResponse(
                {"status": "not_found"},
                status=404,
            )

        if product.stock < qty:
            return JsonResponse(
                {"status": "out_of_stock"},
                status=409,
            )

        product.stock -= qty
        product.save(update_fields=["stock"])

    return JsonResponse({"status": "success"})
```

21. How select_for_update() Works

Suppose the stock is 10.

1. Request A locks the product row.
2. Request B attempts to lock the same row and normally waits.
3. Request A checks stock, updates it, and commits.
4. Request A releases the lock.
5. Request B proceeds and checks the updated stock.

This prevents both transactions from independently updating the same row using the same stale value.

22. Why transaction.atomic()?

`select_for_update()` should normally be used inside a transaction.

```
from django.db import transaction

with transaction.atomic():
    product = (
        Product.objects
        .select_for_update()
        .get(id=product_id)
    )
```

`transaction.atomic()` groups database operations into a transaction. If an exception escapes the atomic block, Django rolls back the transaction.

Note: `select_for_update()` behavior depends on the database backend. SQLite does not provide effective row-level locking through this method.

---

23. Solution 2 — F() Expression

For simple stock or counter updates, Django's `F()` expressions are very useful.

```
from django.db.models import F
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from .models import Product


@require_POST
def buy_product(request, product_id):
    try:
        qty = int(request.POST.get("qty", "0"))
    except (TypeError, ValueError):
        return JsonResponse(
            {"status": "invalid_quantity"},
            status=400,
        )

    if qty <= 0:
        return JsonResponse(
            {"status": "invalid_quantity"},
            status=400,
        )

    updated = Product.objects.filter(
        id=product_id,
        stock__gte=qty,
    ).update(
        stock=F("stock") - qty
    )

    if updated:
        return JsonResponse({"status": "success"})

    return JsonResponse(
        {"status": "out_of_stock"},
        status=409,
    )
```

24. How F() Works

The application does not first read stock into Python. Instead, the database performs the update using the current database value.

Conceptually:

```
UPDATE product
SET stock = stock - qty
WHERE id = product_id
  AND stock >= qty;
```

The database evaluates the stock condition as part of the update. Concurrent updates to the same row are coordinated by the database.

This is useful for atomic field updates.

25. select_for_update() vs F()

Feature	**select_for_update()**	**F()**
Main purpose	Row locking	Database-side expressions
Reads object first	Usually yes	Usually no
Explicit row lock	Yes, on supported backends	No explicit lock requested
Complex business logic	Excellent	Limited for read-check-update workflows

Simple stock update	Possible	Excellent
Transaction	Normally used inside `atomic()`	Not always required

Use `F()` for simple atomic updates. Use `transaction.atomic()` with `select_for_update()` when you must read a row, check multiple business rules, and safely update the row.

---

26. Other Race Condition Solutions

1. Database Transactions

```
from django.db import transaction

with transaction.atomic():
    # Database operations
    pass
```

2. Row Locking

```
Product.objects.select_for_update().get(id=product_id)
```

3. Atomic Database Updates

```
from django.db.models import F

Product.objects.filter(id=product_id).update(
    stock=F("stock") - 1
)
```

4. Database Constraints

Use appropriate constraints to prevent invalid states, such as negative stock where applicable.

5. Queue

Some operations can be processed sequentially using task queues such as Celery or RQ, depending on the architecture.

6. Application-Level Lock

```
from threading import Lock

lock = Lock()

with lock:
    # Protected code
    pass
```

A Python `threading.Lock` does not provide a shared lock across multiple Django worker processes or multiple servers. Database-level synchronization is generally more appropriate for shared database state.

27. Database Backend Considerations

`select_for_update()` depends on database transaction and locking capabilities.

For MySQL, a transactional storage engine such as InnoDB is important for transactional locking behavior.

Always test concurrency behavior using the same database engine and configuration used in production.

28. Real-World Race Condition Examples

E-commerce Stock

```
Available stock = 10
Request A buys 7
Request B buys 5
```

Wallet

```
Balance = 1000
Request A withdraws 500
Request B withdraws 500
```

Ticket Booking

Two users attempt to book the same remaining seat.

Coupon Redemption

Multiple users attempt to redeem a coupon with a limited usage count.

Like Counter

```
likes = likes + 1
```

Unsafe read-modify-write logic can cause lost updates.

29. Race Condition vs Deadlock

Race Condition

- Concurrent operations interfere with one another.
- Results may depend on timing.
- Incorrect or unexpected results may occur.

Deadlock

- Transaction A holds Lock X and waits for Lock Y.
- Transaction B holds Lock Y and waits for Lock X.
- Both transactions may remain blocked until the database detects and resolves the deadlock.

A race condition is a concurrency correctness problem; a deadlock is a circular waiting problem.

---

30. Authentication vs Authorization vs Permission

This is important for interviews.

```
Login
  |
Authentication
  |
Identify User
  |
Check Permission
  |
Authorization
  |
Allow or Deny
```

Example:

```
request.user.has_perm("blog.can_publish")
```

This checks whether the user has the specified permission.

31. Final Mental Model

Django Permission System

```
User
 |
Authentication
 |
User / Group Permissions
 |
has_perm()
 |
Allow / Deny
```

Django Race Condition

```
Multiple Requests
 |
Same Database Row
 |
Concurrent Read / Write
 |
Race Condition
 |
Solutions:
- transaction.atomic()
- select_for_update()
- F()
- Database Constraints
- Appropriate Atomic Operations
```

---

32. Interview Quick Revision

Q1. What is Permission?

Permission determines whether a user can perform a specific action.

Q2. What are Django's default model permissions?

`add`, `change`, `delete`, and `view`.

Q3. How do you check a permission?

```
user.has_perm("blog.can_publish")
```

Q4. How do you protect an FBV?

```
@permission_required("blog.can_publish")
```

Q5. How do you protect a CBV?

Use `PermissionRequiredMixin`.

Q6. What is a Group?

A group allows multiple users to share a set of permissions.

Q7. What is a Race Condition?

A concurrency problem where the timing or order of operations causes incorrect or unexpected results when shared data is accessed or modified.

Q8. How can you prevent race conditions in Django?

- `transaction.atomic()`
- `select_for_update()`
- `F()` expressions
- Database constraints
- Atomic database operations
- Queues where appropriate

Q9. What is select_for_update()?

A Django ORM method that requests row-level locking for selected rows within a transaction on supported database backends.

Q10. What is F()?

A Django ORM expression that allows database-side operations using the current value of a model field.

Q11. Is runserver a production server?

No. Django's development server is intended for development, not production deployment.

Q12. Is hiding a button enough for permission security?

No. Permissions must be enforced on the backend view or API.

Final takeaway: Django permissions protect actions, while concurrency-control techniques protect shared data from conflicting simultaneous operations.